Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory Work 1 & 2:
The OS as a Resource Manager
Track A (Linux) and Track B (xv6)

Author:
st. gr. FAF-242 Tacu Igor

Verified:
asist. univ.

Chişinău – 2026

Contents

OBJECTIVE **2**

PART I — Lab 1: The OS as a Resource Manager (Track A, Linux) **2**

1. Files and Directories 2
2. Processes 5
3. Memory 7
4. Devices and Storage 8

Lab 1 — Synthesis 9

PART II — Lab 2: Meet the OS You Will Build (Track B, xv6) **9**

1. Build It and Boot It 9
2. Use xv6 as the Unix It Is 10
3. Read the Source 11
4. Write My First xv6 Program 14

Lab 2 — Synthesis 16

CONCLUSION **16**

OBJECTIVE

The objective of these two laboratory works is to see, first hand, how an operating system manages the four resources introduced in the lecture: files, processes, memory, and devices. Lab 1 (Track A) approaches this from the outside, observing a real Linux virtual machine through standard commands. Lab 2 (Track B) approaches it from the inside, by building a small teaching operating system, xv6, from source, using it, reading its kernel/user split, and extending it with a new program.

PART I — LAB 1: THE OS AS A RESOURCE MANAGER (TRACK A, LINUX)

All work for this part was carried out as `root` on a Linux VM (`ubuntu-4gb-hel1-1`), inside a working folder `~/os-lab1`.

1. Files and Directories (File Management)

I started by looking around the filesystem tree with `pwd`, `ls`, and `ls -la /`, then checked my home directory with `ls -la ~` and peeked into `/etc`. After that I created a `demo` folder and put a test file through the create, copy, rename, and remove cycle, then checked and changed its permission bits with `chmod`.

Listing 1: Commands run for file management

```
pwd
ls -la /
ls -la ~
cd /etc/ && ls | head

mkdir demo && cd demo
echo "Hello Operating System" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
ls -l note.txt
chmod 600 note.txt
ls -l note.txt
chmod 644 note.txt
ls -l note.txt
```

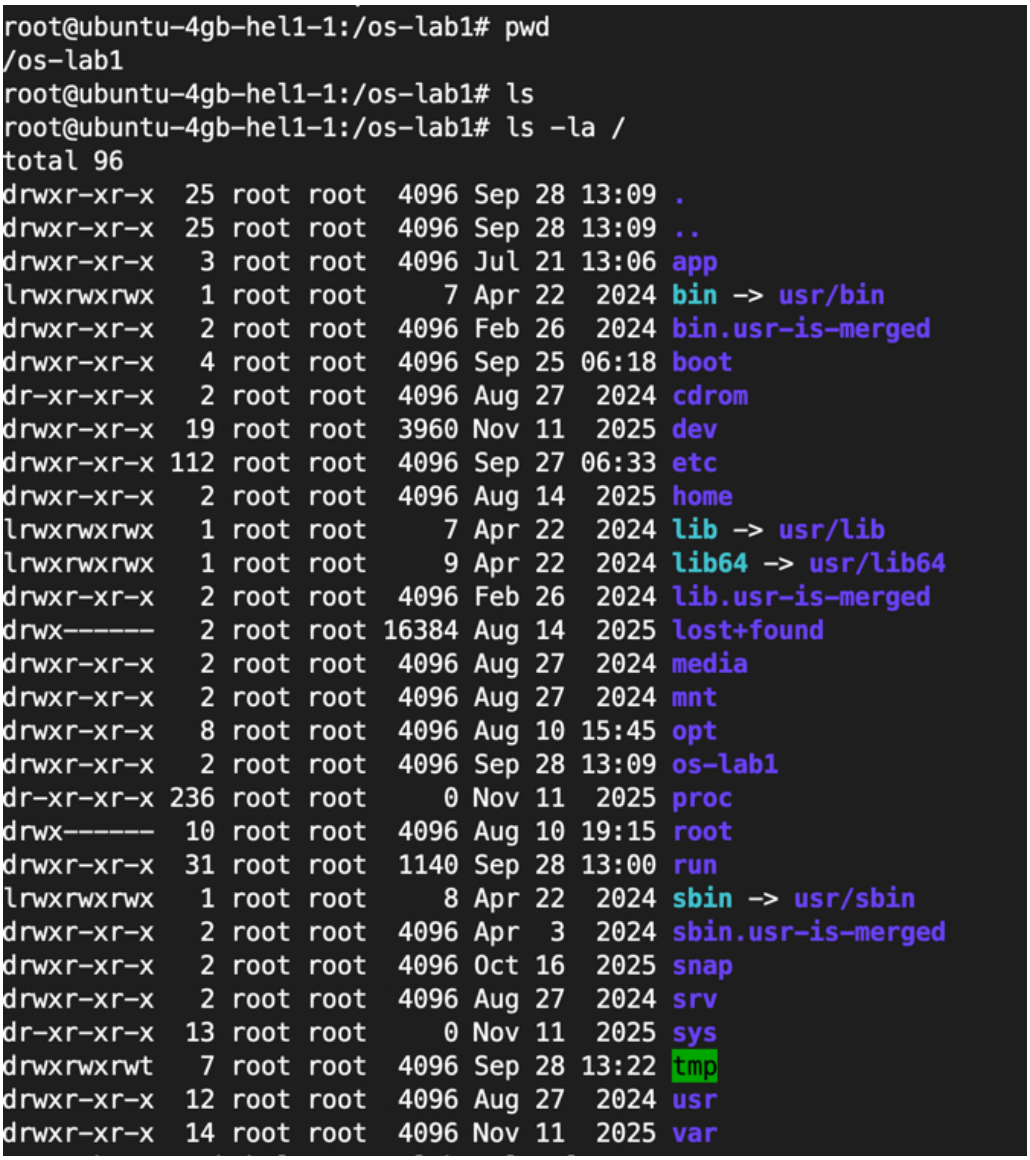


Figure 1: `pwd`, `ls`, and `ls -la /` on the VM.

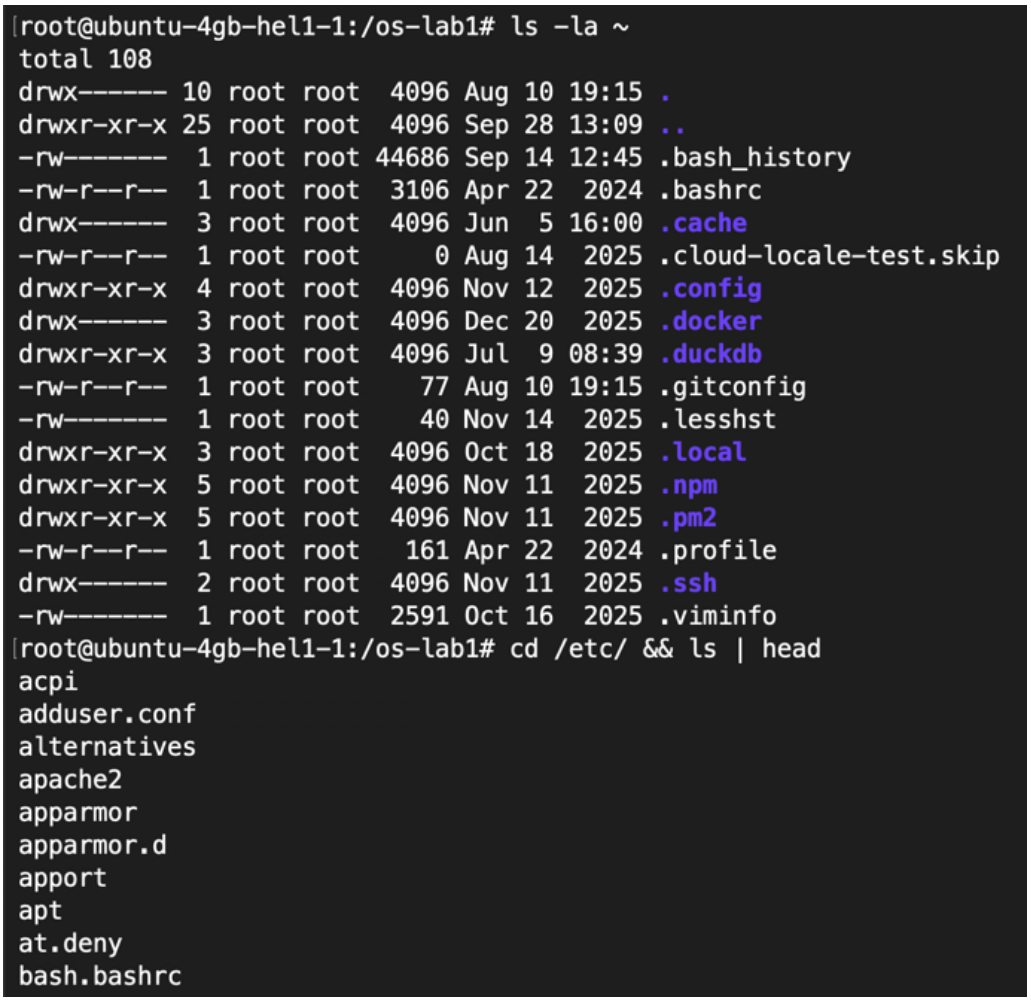


Figure 2: `ls -la ~` and `cd /etc && ls | head`.

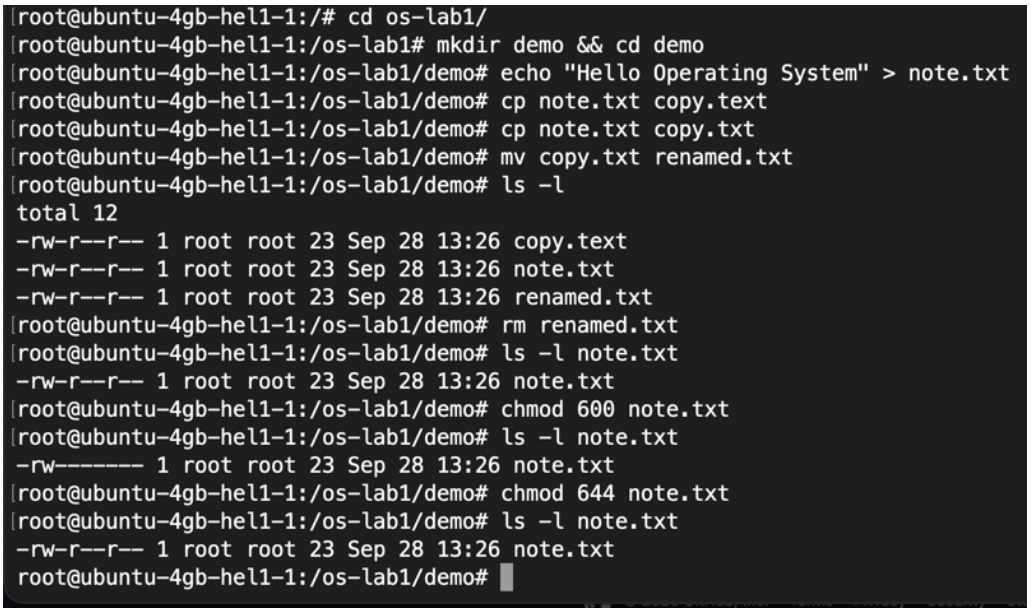


Figure 3: Create, copy, rename, remove, and `chmod` cycle in `demo/`.

Observations:

- **Who owns the files just created, and which group?** `root:root`. Since I was logged in and working as the `root` user, every file I created (`note.txt`, `copy.text`, `renamed.txt`) is owned by user `root` and group `root`, as shown in the `ls -l` owner/group columns.
- **What do the ten characters at the start of `ls -l` mean for `note.txt` after `chmod 600`?** `-rw-------`: the first `-` marks it as a regular file; the next three (`rw-`) are the owner's permissions (read + write, no execute); the next three (`---`) are the group's permissions (none); the last three (`---`) are everyone else's permissions (none). So only `root` can read or write the file, not even users in the `root` group.
- **Name two directories directly under / and say what they hold.** `/etc` holds system-wide configuration files (e.g. `adduser.conf`, `apt`, `bash.bashrc`). `/root` is the home directory of the root user, where my shell's dotfiles (`.bashrc`, `.ssh`, `.profile`, etc.) live.

2. Processes (Process Management)

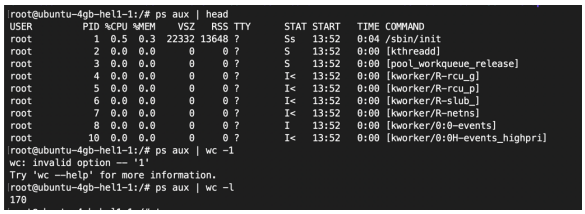
I listed the running processes with `ps aux | head` and counted them with `ps aux | wc -l`, then opened `top` to watch CPU and memory usage live. Next I put `sleep 300` in the background, found it again with `jobs` and `ps -ef | grep sleep`, and killed it by PID. After that I launched another background `sleep`, looked at everything the kernel exposes about it under `/proc/$PID/`, and read its `status` file before killing it too.

Listing 2: Commands run for process management

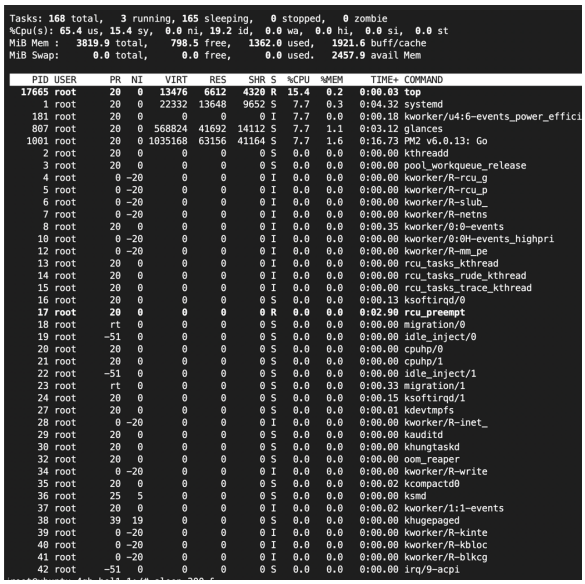
```
ps aux | head
ps aux | wc -l
top

sleep 300 &
jobs
ps -ef | grep sleep
kill <PID>

sleep 300 &
PID=$!
ls /proc/$PID
cat /proc/$PID/status | head -20
kill $PID
```



(a) `ps aux | head` and process count.



(b) Live view with `top`.

Figure 4: Listing and observing processes on the VM.



(a) Background `sleep`, `jobs`, `ps -ef`, `kill`.



(b) `/proc/$PID` exploration and `status`.

Figure 5: Background process control and `/proc` inspection.

Observations:

- **What is the PID of process number 1, and what is it?** PID 1 is `/sbin/init`, the first process the kernel starts at boot, which then spawns and supervises (reaps) every other process on the system.
- **Roughly how many processes were running on the idle VM?** About 170 processes/threads according to `ps aux | wc -l` (168 tasks reported by `top`, of which 3 running and 165 sleeping).
- **What does the "State:" line say for the sleeping process?** `State: S (sleeping)`. The process is blocked waiting on an event (here, the `sleep` timer) and is not consuming CPU while it waits.

3. Memory (Memory Management)

I checked the overall RAM/swap totals with `free -h` and `/proc/meminfo`, then started one more background `sleep 300`, grabbed its PID, and read the `VmRSS` line from `/proc/$PID/status` to see how much physical RAM a bare `sleep` process actually uses.

Listing 3: Commands run for memory management

```
free -h
cat /proc/meminfo | head -6

sleep 300 &
PID=$!
grep VmRSS /proc/$PID/status
kill $PID
```

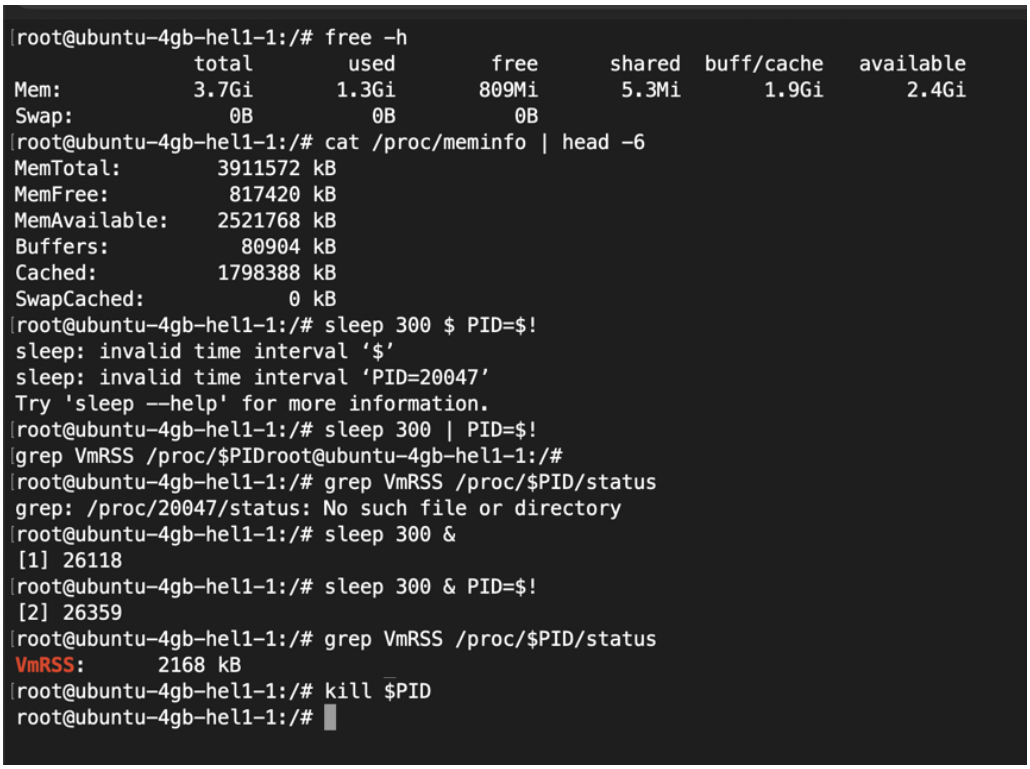


Figure 6: `free -h`, `/proc/meminfo`, and `VmRSS` of a `sleep` process.

Observations:

- **How much total RAM does the VM have, and how much is free?** The VM has 3.7 GiB (≈3,911,572 kB) of total RAM, with 809 MiB reported as immediately free and ∼2.4 GiB "available" once reclaimable buffer/cache is counted.
- **What is swap, and how much is configured?** Swap is disk space the kernel can use as an overflow for RAM when physical memory runs low, swapping out

inactive pages. On this VM, swap is `0B`. None is configured, so this instance has no fallback memory beyond physical RAM.

- **How much resident memory (VmRSS) does a bare `sleep` process use?** Only about 2168 kB (∼2.1 MB). That is small, but not surprising: `sleep` is a tiny, statically simple binary that just calls `nanosleep()` and blocks, so it needs almost no working memory beyond its loaded code/stack pages.

4. Devices and Storage (I/O Management)

I checked the mounted filesystems and free space with `df -h`, looked at the underlying block devices with `lsblk`, measured my lab folder's size with `du -sh`, listed device files under `/dev`, and sampled the live mount table with `mount | head`.

Listing 4: Commands run for devices and storage

```
df -h
lsblk
du -sh ~/os-lab1
ls -l /dev | head
mount | head
```

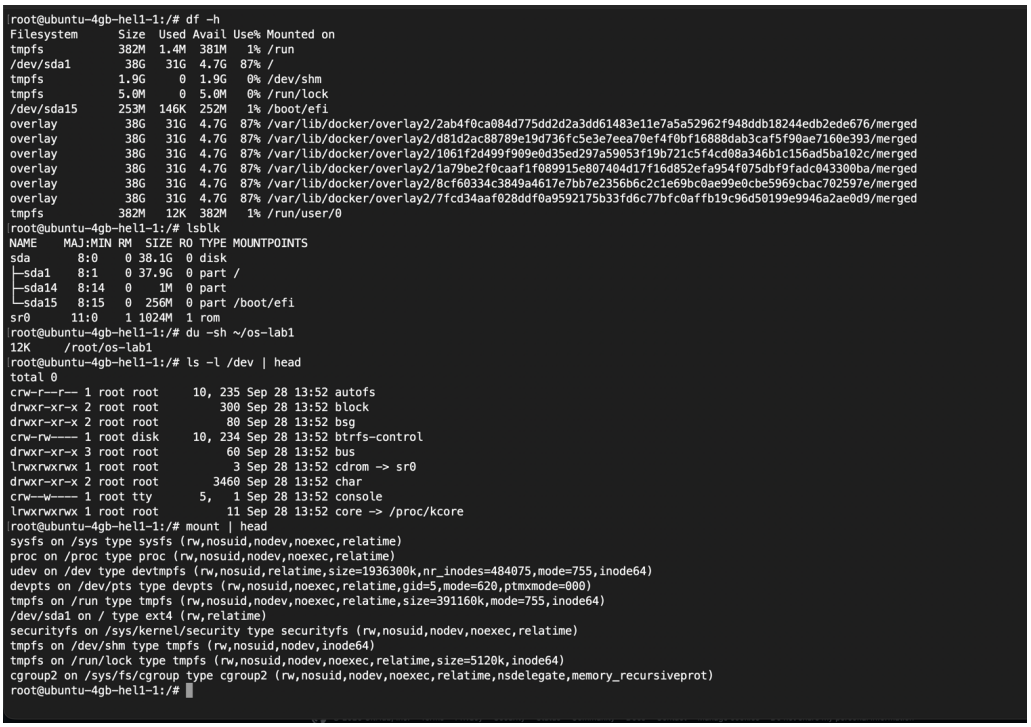


Figure 7: `df -h`, `lsblk`, `du -sh`, `/dev`, and `mount`.

Observations:

- **Which device is the root filesystem "/" mounted on?** `/dev/sda1`, an `ext4` partition (37.9 GiB), as shown by both `df -h` and `lsblk` and confirmed by `mount` (`/dev/sda1 on / type ext4`).
- **Give one entry from `/dev` and what it stands for.** `/dev/console` is a character device (`crw--w----`, major 5, minor 1) representing the system console, the device the kernel writes boot/system messages to and that early init treats as "the terminal."
- **What does "everything is a file" mean, based on what was observed?** It means the kernel exposes hardware and system objects (disks like `/dev/sda1`, the optical drive via `/dev/cdrom` as a symlink to `sr0`, the console, even kernel memory through `/dev/core` linking to `/proc/kcore`) as ordinary-looking file paths under `/dev`, so the same tools (`ls`, `cat`, permissions, `read`/`write`) that work on a text file also work on a physical device.

Lab 1 — Synthesis

Each part of this lab put one of the OS's four core resources in front of me. Files: the kernel presents disk storage as a tree of named, owned, permissioned files, and I could explore and edit that with `ls -l`, `chmod`, `cp`/`mv`/`rm`. Processes: every running program is something the OS created, schedules, and can kill, which `ps aux` and `kill` made visible. Memory: the OS splits physical RAM between all processes and keeps track of each one's footprint, which showed up through `free -h` and the `VmRSS` line in `/proc/<pid>/status`. Devices and I/O: disks, terminals, and other hardware all sit behind the same file-like interface under `/dev`, which `lsblk` and `ls -l /dev` made concrete.

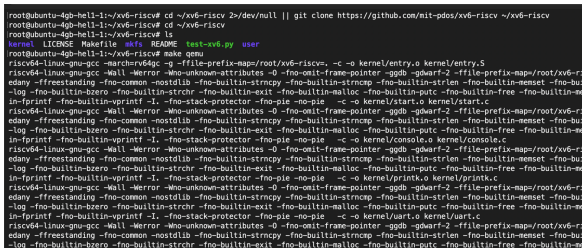
PART II — LAB 2: MEET THE OS YOU WILL BUILD (TRACK B, XV6)

1. Build It and Boot It

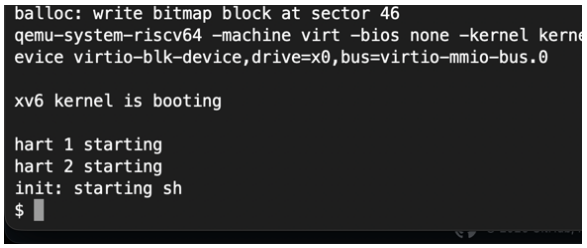
I cloned xv6 into `~/xv6-riscv` and ran `make qemu`. The first attempt failed immediately: `Couldn't find a riscv64 version of GCC/binutils`, and `qemu-system-riscv64: not found`. The VM did not have the RISC-V cross-toolchain or QEMU installed at all, so `make` fell back to the system's regular `gcc` and tried to pass it `-march=rv64gc`, which is not a valid flag for x86 gcc. I installed the missing pieces (`gcc-riscv64-linux-gnu`, `binutils-riscv64-linux-gnu`, `qemu-system-misc`), ran `make clean` to discard the object files built with the wrong compiler, and re-ran `make qemu`. This time it compiled cleanly through the whole kernel and dropped me into the xv6 shell.

Listing 5: Commands run to build and boot xv6

```
cd ~/xv6-riscv 2>/dev/null || git clone https://github.com/mit-pdos/xv6
    -riscv ~/xv6-riscv
cd ~/xv6-riscv
ls
make qemu
```



(a) Cloning the repository and starting the build.



(b) xv6 booted, dropped into the shell.

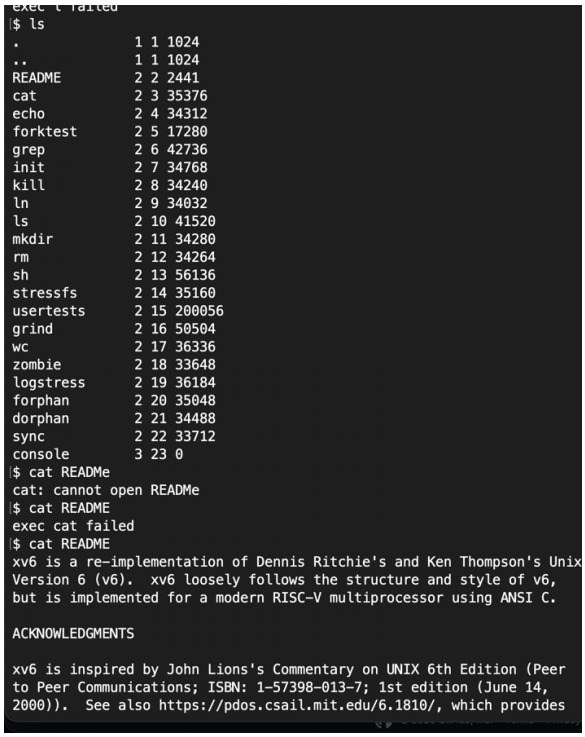
Figure 8: Building and booting xv6 in QEMU.

2. Use xv6 as the Unix It Is

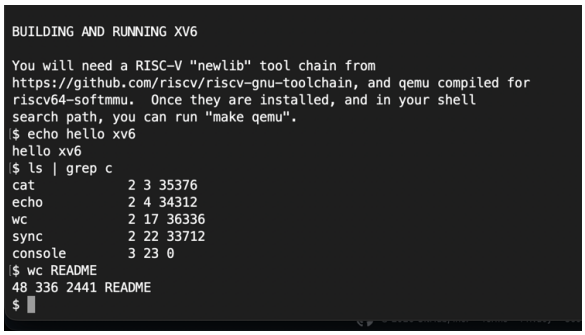
At the `$` prompt I ran `ls` to see what programs xv6 ships with, `cat README` to read its own description of itself, `echo hello xv6` to check the basics, `ls | grep c` to confirm pipes work, and `wc README` to count lines/words/characters in the readme.

Listing 6: Commands run inside the xv6 shell

```
ls
cat README
echo hello xv6
ls | grep c
wc README
```



(a) `ls` and `cat README`.



(b) `echo`, a pipe, and `wc`.

Figure 9: Using xv6's built-in Unix utilities.

Observations:

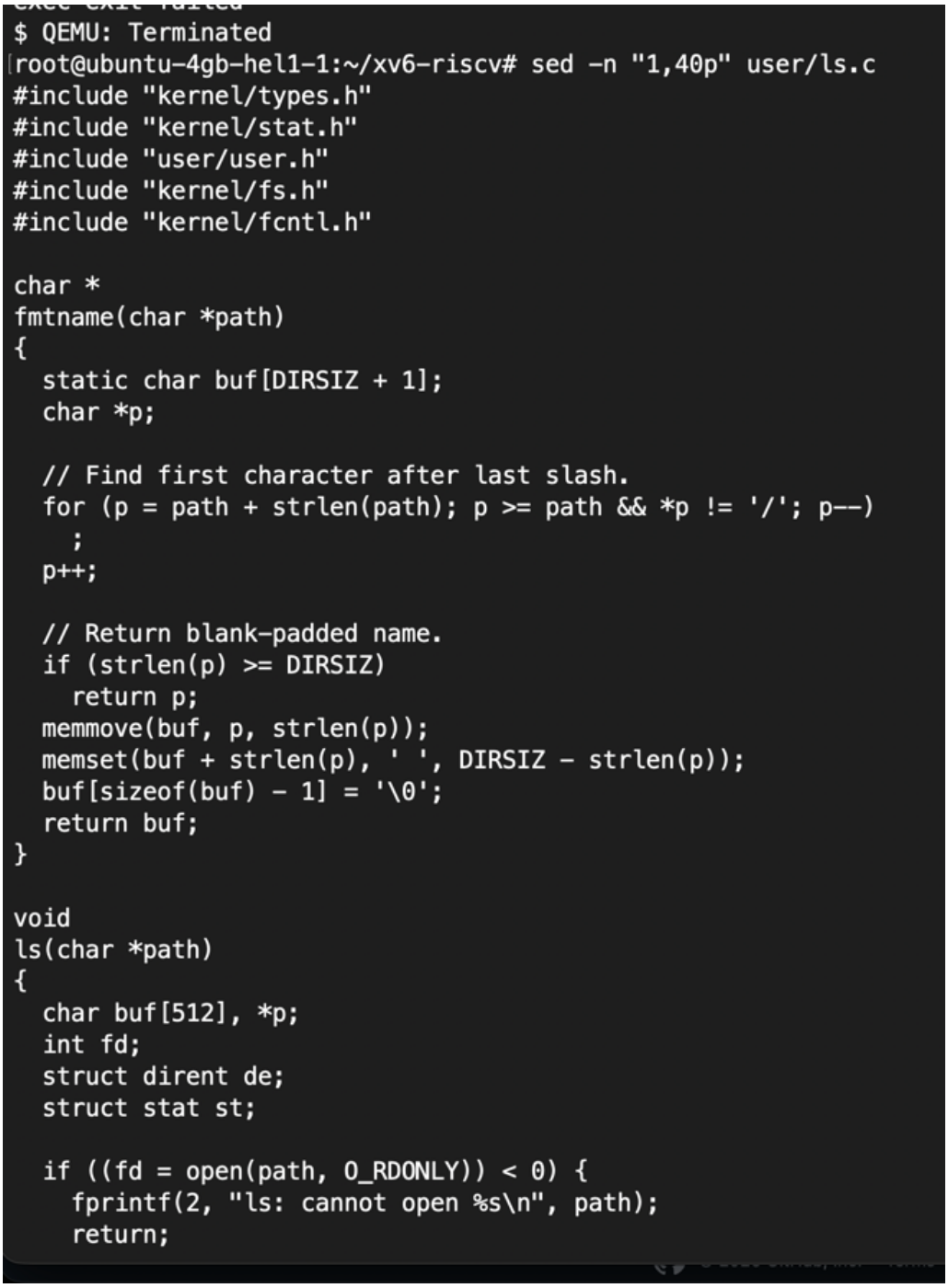
- **List three programs xv6 ships with.** From the `ls` output: `cat`, `echo`, `ls` itself, plus others like `grep`, `wc`, `sh`, `init`, `kill`, `mkdir`, `rm`, `usertests`.
- **Which two OS features must exist for a pipe to work?** Processes and inter-process communication. `ls | grep c` needs two separate processes running at once (the OS's process management), and a channel connecting one process's stdout to the other's stdin (the OS's pipe/IPC mechanism) so data can flow between them without a file on disk.
- **How does the xv6 shell compare to the Linux shell from Lab 1?** It is the same idea stripped down to the essentials: xv6's `sh` handles commands, pipes, and redirection just like bash does, just without bash's scripting features, job control, or the hundreds of built-ins Linux accumulated over the decades.

3. Read the Source

I exited QEMU (`Ctrl-A`, then `X`) and read through `user/ls.c` and `user/cat.c` with `sed`. Then I opened `user/user.h`, which lists every system call a user program is allowed to make, and grepped `kernel/sysfile.c` for `sys_read`/ `sys_write` to see where those calls are actually implemented on the kernel side.

Listing 7: Commands run to read the xv6 source

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```



```
$ QEMU: Terminated
root@ubuntu-4gb-hel1-1:~/xv6-riscv# sed -n "1,40p" user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
  buf[sizeof(buf) - 1] = '\0';
  return buf;
}

void
ls(char *path)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "ls: cannot open %s\n", path);
    return;
```

Figure 10: `user/ls.c` source.



```
  if (fstat(fd, &st) < 0) {
root@ubuntu-4gb-hel1-1:~/xv6-riscv# sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}

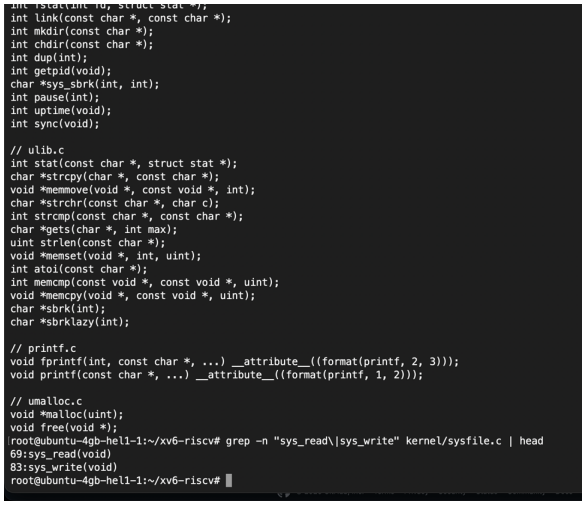
int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
root@ubuntu-4gb-hel1-1:~/xv6-riscv# cat user/user.h
#define SBRK_ERROR ((char *)-1)

struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
```

(a) `user/cat.c` and the start of `user.h`.



```
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);

// ulib.c
int stat(const char *, struct stat *);
char *strcpy(char *, const char *);
void *memmove(void *, const void *, int);
char *strchr(const char *, char c);
int strcmp(const char *, const char *);
char *gets(char *, int max);
uint strlen(const char *);
void *memset(void *, int, uint);
int atoi(const char *);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char *sbrk(int);
char *sbrklazy(int);

// printf.c
void fprintf(int, const char *, ...) __attribute__((format(printf, 2, 3)));
void printf(const char *, ...) __attribute__((format(printf, 1, 2)));

// umalloc.c
void *malloc(uint);
void free(void *);
root@ubuntu-4gb-hel1-1:~/xv6-riscv# grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

(b) Rest of `user.h` and the `sys_read`/`sys_write` grep.

Figure 11: Reading the xv6 user-side source and syscall interface.

Observations:

- **Which system calls does `user/cat.c` use, and what does each one ask the kernel to do?** `read()` and `write()`. `read(fd, buf, sizeof(buf))` asks the kernel to pull bytes from a file descriptor into a buffer, and `write(1, buf, n)` asks it to push bytes from that buffer out to file descriptor 1 (standard output). `cat` itself is just a loop that keeps calling `read` then `write` until there is nothing left to read.
- **In which kernel file and line is `sys_read` implemented?** `kernel/sysfile.c`, line 69 (`sys_write` sits right below it, at line 83).
- **What is the difference between the code in `kernel/` and the code in `user/`?** Code in `kernel/` runs privileged and can touch hardware and other processes directly, while code in `user/` runs unprivileged and can only ask the kernel to do things on its behalf, through system calls like the ones declared in `user.h`.

4. Write My First xv6 Program (user/sleep.c)

I wrote user/sleep.c following the classic xv6 pattern: check argc, print a usage message and exit(1) if it is wrong, otherwise call the sleep syscall and exit(0). My first draft called sleep(atoi(argv[1])), the way the lab sheet shows it.

Adding it to the build was not quite as smooth as expected. First I added $U/_sleep\ to the UPROGS list in the Makefile, but on the first rebuild the binary still was not there and running sleep 10 inside xv6 gave exec sleep failed. The Makefile edit had not actually landed, so I redid it more carefully and confirmed with grep -A 30 "UPROGS=" Makefile that $U/_sleep\ was really in the list this time.

That fixed the "not found" problem, but then the build itself broke: gcc reported implicit declaration of function 'sleep'. Checking user/user.h and user/usys.pl, the two files that together expose a syscall to user programs, showed neither one mentioned sleep at all. Checking the kernel side (kernel/syscall.h, kernel/syscall.c) showed why: this particular xv6 tree does not have a syscall called sleep. It has one called pause (SYS_pause, sys_pause) that does exactly what the classic sleep syscall does, just under a different name. So I changed the call in my program from sleep(atoi(argv[1])) to pause(atoi(argv[1])), keeping the file named sleep.c and the usage message unchanged, since the lab only cares about the program's behavior, not the syscall's internal name. After that, make qemu built clean and both test cases worked.

Listing 8: Final version of user/sleep.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // system call into the kernel
                        // (this xv6 tree renamed "sleep" to "pause")
  exit(0);
}
```

Listing 9: Commands used to build and test sleep

```
make qemu
sleep 10
sleep
```

```
root@ubuntu-4gb-hel1-1:~/xv6-riscv# cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1])); // system call into the kernel
  exit(0);
}
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

(a) First draft of `sleep.c`, still calling `sleep()`.

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_forktest\
        $U/_grep\
        $U/_init\
        $U/_kill\
        $U/_ln\
        $U/_ls\
        $U/_mkdir\
        $U/_rm\
        $U/_sh\
        $U/_stressfs\
        $U/_usertests\
        $U/_grind\
        $U/_wc\
        $U/_zombie\
        $U/_logstress\
        $U/_forphan\
        $U/_dorphan\
        $U/_sync\
```

(b) `UPROGS` list in the Makefile before adding `sleep`.

Figure 12: Debugging the build before the fix.

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1
balloc: first 988 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
evice virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
$ sleep
usage: sleep <ticks>
$
```

Figure 13: `sleep 10` pausing then returning, and `sleep` printing its usage message.

Lab 2 — Synthesis

By the end of this lab I had xv6 built and booted from source, used it as a small Unix, and read both halves of it: the user programs that ask for things, and the kernel code that grants them. Writing `sleep.c` made that split concrete, since getting it working meant tracing a syscall from my program, through `user.h`, into the kernel's syscall table, and finding out its name had changed along the way. It is a small tree, but I could hold the whole path in my head while debugging it, which is exactly the point of using xv6 for this.

CONCLUSION

Lab 1 and Lab 2 felt like two ends of the same rope. In Lab 1 I could only watch Linux from the outside, through `ps`, `/proc`, `ls -l /dev`, and infer what the kernel must be doing behind those interfaces. In Lab 2 I got to see the other side directly: the actual C code that implements a syscall, and what happens when a user program's idea of that syscall does not match the kernel's. The `pause`/`sleep` mismatch in Lab 2 was a good example of something Lab 1 could only hint at: processes ask the kernel for things by name and number, and if the two sides disagree on either, nothing works, no matter how correct the surrounding code looks in isolation.